

Warewe Consultancy Full Stack Developer - What to Learn

Overview

Small-team SaaS shop: Node.js/React/Next.js/TypeScript full-stack, with PostgreSQL + MongoDB, a cache/queue layer (Redis + RabbitMQ), Docker deployment, Stripe billing, and AI/ML agent work. Your React/Next.js/TypeScript/Node/Docker/AI-agent background already covers most of this — the gaps below are additive infrastructure, not a new paradigm.

MUST know before interview

MongoDB (document database)

- What: NoSQL, schema-flexible document store — JSON-like documents in collections, no fixed columns.
- Why: likely used for less-structured data (logs, activity, config) alongside PostgreSQL's relational core.
- Bridge: you write raw SQL (Skimly) and use Prisma (Vibe) already. Mongo swaps joins/schemas for flexible documents, queried with `find()`/`aggregate()` instead of SQL.
- Learn: CRUD via Node driver/Mongoose; embedding vs. referencing; basic `$match/$group` aggregation; indexes.

Redis (in-memory cache / key-value store)

- What: in-memory store for caching, sessions, rate limiting, pub/sub.
- Why: "scalable architectures" for growing SaaS means caching hot reads, offloading session/rate-limit state from the primary DB.
- Bridge: you've optimized load speed before (Clink's 30%) — Redis is the backend analog: cache a query result with a TTL instead of hitting Postgres/Mongo every request.
- Learn: `GET/SET/EXPIRE`, Redis as a session store, basic rate-limiting pattern, why cache invalidation is the hard part.

RabbitMQ (message queue)

- What: message broker for async, decoupled service communication (producer publishes, consumer processes independently).
- Why: async background jobs (emails, webhooks, long AI tasks) get queued instead of blocking requests.
- Bridge: you've built async job patterns with Inngest (Vibe) and webhook-driven events (Stripe/Clerk). Same idea, but you wire the broker/queue/consumer yourself instead of a managed service.
- Learn: producer/consumer/queue/exchange vocabulary, basic direct-exchange routing, why queues improve reliability.

GOOD to know

AI/ML & AI Agents: you already have strong real answers (Echo's RAG + tool-calling agent, Vibe's autonomous coding agent) — be ready to discuss extending an agent to call internal APIs for SaaS automation.

Owning a service end-to-end: the JD wants concept-to-production ownership. Frame Clink AI (EAS builds, OTA updates) and Docker/E2B project deploys as full-cycle ownership evidence.

Not urgent

Angular and Kubernetes are generic keywords not emphasized in this specific JD's core stack — skip deep prep unless it comes up.